Опис коду

Програма реалізує двошарову нейронну мережу для класифікації даних із датасету Iris за допомогою чисельної бібліотеки NumPy. Код виконує стратифікований поділ даних (70/30) із фіксованим генератором випадкових чисел та їх стандартизацію.

Реалізовано прямий прохід з чисельно стабільною крос-ентропією (через log-softmax зі зсувом на максимум). Вручну запрограмовано алгоритм зворотного поширення похибки для розрахунку градієнтів за всіма параметрами (W1, b1, W2, b2).

Для перевірки правильності виконано перевірку: звірку всіх градієнтів та втрат із еталонною реалізацією в PyTorch та чисельне диференціювання.

Також передбачено тестовий режим із навмисною помилкою (вилучення ділення на розмір вибірки N) для перевірки здатності тестів виявляти баги у зворотному проході.

Результати ч 1: Реалізація в NumPy

Реалізовано:

- прямий прохід двох афінних шарів із ReLU;
- чисельно стабільну крос-ентропію через log-softmax зі зсувом на максимум;
- збереження проміжних значень для зворотного проходу;
- ручне обчислення градієнтів за W1,b1,W2,b2.

Формули:

Прямий прохід:

- $Z_1 = X \cdot W_1 + b_1$ — розмірність (105, 8)
- $A_1 = \mathrm{ReLU}(Z_1)$ — розмірність (105, 8)
- $Z_2 = A_1 \cdot W_2 + b_2$ — розмірність (105, 3)
- $P = \mathrm{softmax}(Z_2)$ — розмірність (105, 3)

Зворотний прохід:

- $dZ_2 = \frac{1}{N}(P - Y_{onehot})$ — розмірність (105, 3)
- $dW_2 = A_1^T \cdot dZ_2$ — розмірність (8, 3)
- $db_2 = \sum_{i=1}^{N} dZ_2^{(i)}$ — розмірність (3,)
- $dA1 = dZ_2 \cdot W_2^T$ — розмірність (105, 8)
- $dZ_1 = dA1 \odot (Z_1 > 0)$ — розмірність (105, 8)
- $dW_1 = X^T \cdot dZ_1$ — розмірність (4, 8)
- $db_1 = \sum_{i=1}^{N} dZ_1^{(i)}$ — розмірність (8,)

Результати ч 2: Звірка з PyTorch

Втрата NumPy: 1.4562007801142816

Втрата PyTorch: 1.4562007801142818

Величина	Макс. абсолютна різниця	Перевірку пройдено
Втрата	2.22e-16	True
Градієнт W1	5.55e-17	True
Градієнт b1	3.64e-17	True
Градієнт W2	8.33e-17	True
Градієнт b2	1.39e-16	True

Результати ч 3: Перевірка окремих градієнтів чисельним диференціюванням

Для кожного вибраного параметра :

1. Зберегли його початкове значення.
2. Збільшили лише цей параметр на $\varepsilon = 10^{-6}$ та обчислили втрату L+.
3. Установили значення на ε менше від початкового та обчислили L-.
4. Відновили початкове значення й обчислили чисельну похідну.

Параметр	backward()	Чисельна похідна	Абс. різниця	Пройдено
W1[0,0]	-0.01992018	-0.01992018	6.03e-11	True
b1[0]	-0.07056611	-0.07056611	3.63e-11	True
W2[0,0]	0.13523021	0.13523021	9.22e-11	True
b2[0]	0.08552164	0.08552164	1.46e-10	True

Результати ч 4: Дослід із навмисною помилкою

Тимчасово прибрано ділення на кількість об’єктів у градієнті за логітами.

Прогноз:

- Втрата не зміниться, бо прямий прохід і функція втрат залишаються незмінними.
- Ненульові градієнти зростуть в N = 105 разів, бо ми прибираємо ділення на N.
- Обидві перевірки виявлять помилку через перевищення допустимих порогових значень різниці.

--- Звірка з PyTorch ---

Величина	Макс. абсолютна різниця	Перевірку пройдено
Втрата	2.22e-16	True
Градієнт W1	1.85e+01	False
Градієнт b1	1.56e+01	False
Градієнт W2	3.42e+01	False
Градієнт b2	1.12e+01	False

--- Чисельне диференціювання ---

Параметр	backward()	Чисельна похідна	Абс. різниця	Пройдено
W1[0,0]	-2.09161935	-0.01992018	2.07e+00	False
b1[0]	-7.40944172	-0.07056611	7.34e+00	False
W2[0,0]	14.19917190	0.13523021	1.41e+01	False
b2[0]	8.97977271	0.08552164	8.89e+00	False

Висновки

У ході роботи ми успішно реалізували двошарову нейронну мережу для класифікації ірисів за допомогою бібліотеки NumPy, налаштували стратифікований поділ і стандартизацію даних, а також власноруч запрограмували прямий та зворотний прохід.

Лабораторна допомогла глибше зрозуміти побудову нейронних мереж та суть роботи з градієнтами. Спочатку було складно зрозуміти формули та логіку мереж, але практична реалізація дала змогу розібратися з цим.

Було ознайомлено з методами перевірки та виявлення помилок. Завдяки чисельному диференціюванню та звірці з бібліотекою PyTorch вдалося переконатися в математичній правильності написаних формул і навчитися ефективно виявляти приховані баги через аналіз похибок.

Отримані результати та успішне проходження всіх тестів довели повну працездатність створеної моделі.